

SANJUBHASKAR

PHD SCHOLAR

BLZENR41E26007

SECTION IV: MDP FORMULATION

The gridworld (Environment) is formalized as a Markov Decision Process (MDP) defined by tuple $\{S, A, P, R, \gamma\}$ across a discrete 4×4 spatial grid with 16 total states.

State Space (S) - Discrete 2D coordinates

$$S = \{s_1, s_2, s_3, \dots, s_{16}\} \text{ where } (s_1 \dots s_{16})$$

00	01	02	03
10	11	12	13
20	21	22	23
30	31	32	33

Action Space (A) - $A = \{UP, DOWN, LEFT, RIGHT\}$

Reward Function

R : Stop penalty $R = -1$ for non terminal moves

Terminal state $R = 0$

$(3, 3)$

Optimization: Maximize expected Return
Total

$$G_t = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}$$

from Start state

$s_1(0,0)$ to s_{16}

2) Policy Evaluation

Policy Evaluation computes state value function $V^\pi(s)$ under an equiprobable random policy

using Bellman Expectation Equation ($\gamma = 0.9, \theta = 10^{-4}$)

$$V_k(s) = \sum_{a \in \mathcal{A}} \pi(a|s) \sum_{r, s'} P(s', r | s, a) [\gamma + \gamma V_k(s')]$$

Iteration Maximum change Δ Convergence status

1 1.75 Not Converged

2 1.09 Not Converged

3 0.81 Not Converged

4 0.73 Not Converged

5 0.66 Not Converged

6 0.60

Δ decays geometrically (-0.9 per iteration) once the value signal has propagated to

all states but does not fall below $\theta = 10^{-4}$

until iteration 74. The full run

converged at 74 iterations.

Selected value function V^π for

random policy (entropy)

1 - 8.58 8.26 -7.72 -7.22

2 -8.26 7.72 -6.67 -5.99

3 -7.72 -6.67 -4.26 -0.57

4 -6.67 -4.26 -0.57 0.57

5 -4.26 -0.57 0.57 0.57

6 -0.57 0.57 0.57 0.57

7 0.57 0.57 0.57 0.57

8 0.57 0.57 0.57 0.57

9 0.57 0.57 0.57 0.57

10 0.57 0.57 0.57 0.57

11 0.57 0.57 0.57 0.57

12 0.57 0.57 0.57 0.57

13 0.57 0.57 0.57 0.57

14 0.57 0.57 0.57 0.57

Value Iteration and Optimal policy generation

Value iteration and optimal policy

generation was run with same

$\gamma = 0.9, \theta = 10^{-4}$. It converged in

only 7 iterations ($\Delta \neq 10.0, 9.0, 8.1, 7.25, 6.561, 5.90, 0$) far faster than policy evaluation, because value iteration greedily backs up the max over action instead of averaging over a fixed policy.

State	Optimal Action	State value
S_1	down	1.81
S_2	down	3.12
S_3	down	4.58
S_4	down	6.20
S_5	down	3.12

For optimal policy in 4x4 grid

	0	1	2	3
0	down	down	down	down
1	down	down	down	down
2	down	down	down	down
3	→	→	→	Goal

Task 4 - Optimal Path Analysis
 Three policies were executed from the start state to S_5 :
 1) A random policy
 2) Sampling uniformly from the 4 actions at each step (capped at 100 steps)
 3) A policy obtained by acting greedily with respect to state value function computed in Task 2 (Policy evaluation of the random policy) and (3) Optimal Policy from value Iteration (Task 3).

Policy	Path length	Total Reward	Goal reached
Random	94	-83	Yes
Evaluate	6	5	Yes
Optimal Policy	6	5	Yes

Both the evaluated Policy and the Optimum Policy find the shortest 6 step path ($S_0 \rightarrow S_4 \rightarrow S_6 \rightarrow S_{12} \rightarrow S_{13} \rightarrow S_{15}$)
 (0,0) (1,0) (2,0) (3,0) (3,1) (3,2)
 $S(3,3)$

earning a total reward of 5 steps

$5 \times 2 = 10$ Random policy eventually reaches goal but only after 94 undirected steps.

Questions

1) What is state in GridWorld?

A state is one discrete cell of the grid, identified by its (row, column) position (equivalently a single index $S_0, S_1, \dots, S_{15}$). It fully describes agent's current situation. (For next action only current state matters, no previous action.)

2) What actions can the agent perform?

Four actions are available with every state: Up, Down, Left, and Right. If an action would move the agent off the edge of the grid, the agent simply stays in its current cell (the wall or bump/bounce rule).

3) What is the purpose of Policy Evaluation?

Policy evaluation computes the state-value function.

For a given fixed policy π , i.e. the expected discounted return an agent following that exact policy

would collect from each state. It answers how good is this policy without changing policy itself.

4) How does Value Iteration differ from Policy Evaluation?

Policy evaluation applies the Bellman expectation equation. It averages over the fixed policy's action probabilities to update each state's value.

Value iteration instead applies the Bellman optimality equation. It

every state it takes the maximum over all possible actions so it

simultaneously evaluates and

improves the policy in a single sweep.

As a result Value iteration

converges to the optimum value

function directly rather than merely

evaluating one policy and the

greedy policy with respect to V^*

is guaranteed optimal where as the

greedy policy with respect to single

evaluate V^* is only guaranteed to

be at least as good as π .

5) Bellman Optimality eqn

$$V^*(s) = \max_a \sum_{s'} P(s'|s, a) [R(s, a) + \gamma V^*(s')]$$

Optimal value of a state as the

value of the best action available here.

In this deterministic Grid World it

simplifies to

$$V^*(s) = \max_a [R(s, a) + \gamma V^*(\text{next}(s, a))]$$

It is the fixed point that value iteration repeatedly applies until values stop changing.

6) Which policy reached goal using the fewest steps?

Optimal policy and evaluated policy both reached goal in 6 steps.

Random policy took 94 steps

7) Why is Optimal policy superior to Random policy?

Optimal policy always chooses the action with highest expected discounted return at each state, so it takes shortest path to goal (6 steps, Reward = 5)

Random policy ignores state values entirely and picks actions uniformly, so it wanders (reached in 94 steps, reward = -83)

8) How would obstacles affect optimal path?

Obstacles (blocked cells) remove or penalize certain transitions. Value

iteration would propagate lower

value to states next to an

obstacle and propagate the optimal policy

around it, generally lengthening the shortest path compared to open grid above.

Conclusion:

Assignment implement a 4x4 gridworld and composed 3 ways of acting it. Policy evaluation applies Bellman's expectation equation to compute how good a policy is. Value iteration applies Bellman optimality equation, taking a max over actions at every sweep and converged in only 7 iterations to the true optimal value function.